CSC_51053_EP, Exercise Session 2: Modeling, Relational Algebra and Calculus

Ioana Manolescu, Madhulika Mohanty, Garima Gaur

October 3, 2025

1 Internship management system

We consider a system for the online management and selection of internship by students.

Each internship has an ID, a title, a category, and a description. Each internship is proposed by a supervisor; supervisors have an ID, a name, and an email. Each student has an ID, a name, and an email.

To be visible to the students, an internship must be validated by a professor responsible of the internship programme. Each professor has an ID, a name, and an email. There may be several professors jointly in charge of the internship program; it is sufficient if one of them validates the internship. The system needs to record who validated an internship, when, and an optional comment.

A student may become selected for an internship, which models that the supervisor has agreed to work with this student on the internship. A student can be selected for at most one internship, and for each internship, at most one student can be selected.

An internship is available if it is validated and no student has been selected for the internship.

1. Propose an E-R model for this information system. For the **people** involved in this scenario:

   - We can use for instance one person(ID, name, email, type) entity.
   - *Or* (better) we can use *four* entities: person(ID, name, email, type), student *isA* person, professor *isA* person, supervisor *isA* person. These don't need any specific attributes since (*i*) all their attributes are already present in the parent; (*ii*) their type(s) are indicated in the entity itself. We find this better since it is *more precise*: we will be able to attach different relationships to different kind of people (see below).
   - *Or* we can use three independent entities: professor(ID, name, email, type), student(ID, name, email, type), supervisor(ID, name, email, type). This is not as good, because it repeats some structure, when this could have been avoided, and does not capture the fact that all these are just people.

   For the **internships**, we create an entity internship(iID, title, description, category). For the **relationships**, we create:

   - proposes between supervisor and internship; every internship must participate to it *exactly once* (constraint on the participation of internship to the relationship proposes);
   - validates between professor and internship; every internship participates *at most once* (because one professor's validation is sufficient; also note that some of them are never validated);
   - selectedFor between student and internship; each internship participates to at most one of these, and each student participates to at most one of these.

   If instead of the specialized people entities we just have the people entity, of course, these relationships carry over the people entity.

2. If we want to register a student in the internship program, what does this correspond to in the E-R model? This corresponds to adding one instance to the student entity instance set.

3. In the E-R model, can we model a validation by a professor not yet registered in the system? No. A relationship can only be instantiated if an entity instance is available to play each of the roles of the relationship.

4. Propose a corresponding relational model.

2 Flight information [1]

Consider the following relations with airline flight information:

Flights(*flno*: integer, from: string, to: string, distance: integer, departs: time, arrives: time)
Aircraft(*aid*: integer, aname: string, cruisingrange: integer)
Certified(*eid*: integer, *aid*: integer)
Employees(*eid*: integer, ename: string, salary: integer)

Employees contain pilots but also other kinds of employees. In Certified, *eid* is a foreign key to Employees and *aid* a foreign key to Aircraft. Certified(e, a) means that employee e is certified to fly (pilot) aircraft a. Employees are pilots if and only if they are certified on at least one aircraft; a pilot may be certified for several aircrafts.

Write the following queries in relational algebra and tuple relational calculus:

1. Find the *eid*s of pilots certified for some Boeing aircraft.

   - $\pi_{Employees.eid}(\sigma_{aname='Boeing\%'}(\text{Aircraft}) \bowtie_{aid} \text{Certified} \bowtie_{eid} \text{Employees})$
     or, relying on a foreign key constraint:
     π_{eid} (Certified $\bowtie$ ($\sigma_{aname='Boeing\%'}$ (Aircraft)))
   - {P | ∃ C ∈ Certified ∃ A ∈ Aircraft ∃ E ∈ Employees
     (C.aid=A.aid ∧ C.eid=E.eid ∧ A.aname='Boeing%' ∧ P.eid=E.eid) }
     or, relying on the foreign key constraint:
     { P | ∃ C ∈ Certified ∃ A ∈ Aircraft (P.eid=C.eid ∧ C.aid=A.aid ∧ A.aname='Boeing%') }

2. Find the names of pilots certified for some Boeing aircraft.

   - $\pi_{Employees.ename}(\sigma_{aname='Boeing\%'}(\text{Aircraft}) \bowtie_{aid} \text{Certified} \bowtie_{eid} \text{Employees})$
   - { P | ∃ C ∈ Certified ∃ A∈ Aircraft ∃ E ∈ Employees (C.eid=E.eid ∧ C.aid=A.aid ∧ A.aname='Boeing%' ∧ P.ename=E.ename)}

3. Find the *aid*s of all aircraft that can be used on non-stop flights from Boston to Madras.

   - π_{aid} ($\sigma_{distance \leq cruisingrange}$ (Aircraft × ($\sigma_{from='Boston' \ AND \ to='Madras'}$ (Flights))))
   - { P | ∃ A ∈ Aircraft ∃ F ∈ Flights (P.aid=A.aid ∧ F.distance ≤ A.cruisingrange ∧ F.to='Madras' ∧ F.from='Boston') }

4. Identify all the flights that can be piloted by every pilot whose salary is more than $100,000.

   - $\pi_{flno,eid}$ ($\sigma_{distance \leq cruisingrange}$ (Flights × (Aircraft ⋈ Certified))) /
     π_{eid} ($\sigma_{salary>100000}$ (Certified ⋈ Employees))
   - The final result is R_3 but we use the intermediate relations R_1 and R_2 for clarity.
     - R_1 = { R | ∃ F ∈ Flights ∃ C ∈ Certified ∃ A ∈ Aircraft (R.eid=C.eid ∧ R.flno=F.flno ∧ F.distance ≤ A.cruisingrange ∧ C.aid=A.aid) };
       R_1 stores the pairs (eid, flno) such that the employee can fly that flight.
     - R_2 = { S | ∃ C ∈ Certified ∃ E ∈ Employees (S.eid=E.eid ∧ E.salary>100000 ∧ C.eid = E.eid)) };
       R_2 stores the IDs of *pilots* paid above 100000.
     - R_3 = { U | ∀ P ∈ R_2 (∃ RI ∈ R_1 (U.flno=RI.flno ∧ RI.eid=P.eid)) }
       R_3 returns all the flights that can be flown by all the pilots paid above 100000.

5. Find the names of pilots who can operate planes with a range greater than 3,000 miles but are not certified on any Boeing aircraft.

   - $\pi_{ename}(\pi_{eid,ename}(\text{Employee} \bowtie_{eid} (\sigma_{cruisinrange>3000}(\text{Aircraft})) \bowtie_{aid} \text{Certified}) - (\pi_{eid,ename}(\text{Employee} \bowtie_{eid} (\sigma_{aname='Boeing\%'}(\text{Aircraft})) \bowtie_{aid} \text{Certified}))$

- { P | ∃ E ∈ Employee, ∃ C_1 ∈ Certified, ∃ A ∈ Aircraft (E.eid = C_1.eid ∧ C_1.aid = A.aid ∧ (A.cruisinrange > 3000) ∧
  ¬ (∃ C_2 ∈ Certified, A_2 ∈ Aircraft (C_1.eid=C_2.eid ∧ A_2.aid=C_2.aid ∧ A_2.name='Boeing%'))) ∧ P.ename=E.ename }

6. Find the *eid*s of employees who make the highest salary.

   - π_{eid} (Employees) −
     $\pi_{E1.eid}$ ($\sigma_{E1.salary<E2.salary}$ (ρ_{E1}(Employees) × ρ_{E2}(Employees)))
   - { R | ∃E_1 ∈ Employee ∧ ¬(∃ E_2 ∈ Employee ((E_1.eid≠ E_2.eid) ∧ (E_2.salary > E_1.salary))∧ E_1.eid=R.eid }. Observe that E_1.eid≠ E_2.eid above is redundant since eid is a key so it is not possible to have E_1.eid=E_2.eid and at the same time E_2.salary > E_1.salary.

7. Find the *eid*s of employees who make the second highest salary.

   - Call the previous query Q_6. We define the query A= π_{eid}(Employees) − Q_6. These are the employees which *do not* have the highest salary. Then, the employees we are looking for are those that have the highest salary in the relation:

     $$B= \pi_{eid,name,salary}(\text{Employees} \bowtie_{eid} A)$$

     To find them, use the algebraic expression of point 6 where **Employee** is replaced with **B**.
   - We want employees whose salary is lower than *one* salary (paid to one or more top-salary employees), but is not lower than *two distinct salaries*. Thus:
     { P | ∃ E_1 ∈ Employee ∃ E_2 ∈ Employee (E_1.sal < E_2.sal ∧
     ¬ (∃ E_3 ∈ Employee ((E_3.sal <> E_2.sal) ∧ (E_3.sal > E_1.sal)))) ∧ P.id=E_1.id }

8. Find the *eid*s of employees who are certified for exactly three aircraft.

   - $\pi_{C1.eid}(\sigma_{C1.aid\neq C2.aid\neq C3.aid\neq C1.aid}(\rho_{C1}(\text{Certified})) \bowtie_{eid} (\rho_{C2}(\text{Certified})) \bowtie_{eid} (\rho_{C3}(\text{Certified})))$
     −
     $\pi_{C1.eid}$ ($\sigma_{\bigwedge_{1\leq i,j\leq 4, i\neq j}(Ci.aid\neq Cj.aid)}(\rho_{C1}(\text{Certified}) \bowtie_{eid} \rho_{C2}(\text{Certified}) \bowtie_{eid} \rho_{C3}(\text{Certified}) \bowtie_{eid}$
     $\rho_{C4}(\text{Certified})))$
   - {C | ∃ C_1, C_2, C_3 ∈ Certified (C.eid=C_1.eid=C_2.eid=C_3.eid ∧ $\bigwedge_{1\leq i,j\leq 3, i\neq j}$($C_i$.aid ≠ C_j.aid)
     ∧ ¬ (∃ C_4 ∈ Certified (C_4.eid = C_1.eid ∧ $\bigwedge_{1\leq i,j\leq 4, i\neq j}$($C_i$.aid ≠ C_j.aid))))}

3 Cardinalities of relational expressions [1]

We denote by $|R|$ the number of tuples in relation R. Given two relations R_1, R_2, give the minimum and maximum number of tuples which may belong to the results of the following relational algebra expressions; also, specify all constraints which should apply for each expression to be correct. (1) $R_1 \cup R_2$; (2) $R_1 \cap R_2$; (3) $R_1 \setminus R_2$; (4) $|R_1| \times |R_2|$; (5) $\sigma_{a=5}(R_1)$; (6) $\pi_a(R_1)$; (7) R_1/R_2.

- $\max(|R_1|, |R_2|) \leq |R_1 \cup R_2| \leq (|R_1| + |R_2|)$; the lower bound is reached when one of the tables is empty, or included in the other; the upper bound is reached when the tables are disjoint.
- $0 \leq |R_1 \cap R_2| \leq \min(|R_1|, |R_2|)$; the lower bound is reached when the tables are disjoint; the upper bound is reached when one of the tables is included into the other.
- $0 \leq |R_1 \setminus R_2| \leq |R_1|)$; the lower bound is reached when R_1 is included in R_2; the upper bound is reached when R_2 is empty.
- $|R_1 \times R_2| = R_1 * R_2$
- $0 \leq |\sigma_{a=5}(R_1)| \leq |R_1|$; the lower bound is reached when there is no tuple with value 5 at its attribute a; the upper bound is reached when all the tuples in R_1 have the value of the attribute a equal to 5.
- $0 \leq \pi_a(R_1)| \leq |R_1|$; the lower bound is reached when R_1 is empty; the upper bound is reached when all the values of attribute a in the tuples of R_1 are distinct.

- Let A and B be the sets of attributes R_1 and R_2. $0 \leq |R_1/R_2| \leq |R_1|$; $0 \leq |R_1/R_2|$: the lower bound is reached when for each tuple t_1 in $\pi_{A-B}(R_1)$, there exists a tuple t_2 in R_2, such that $t_1.t_2$ is not in R_1. The upper bound is $|R_1|/|R_2|$: this is reached when for each tuple t_1 in $\pi_{A-B}(R_1)$, for each tuple t_2 in R_2, $t_1.t_2$ is in R_1. A (wider) uper bound is easily derived from this: when $|R_2| = 1$, $|R_1/R_2| = |R_1|$.

4 Expressivity

1. Show how to express intersection ($\cap$) using only operators among: select (σ), project (π), union ($\cup$), cartesian product ($\times$) and set difference ($\setminus$).

   $R \cap S = R \setminus (R \setminus S)$

2. Show how to express relational division ($/$) using only operators among: select (σ), project (π), union ($\cup$), cartesian product ($\times$), and set difference ($\setminus$).

   Let $R(\bar{a}, \bar{b})$ and $S(\bar{b})$ be two relations where $\bar{a}$, $\bar{b}$ are attribute sets.

   (a) $e_1 \leftarrow \pi_{\bar{a}}(R) \times S$: these are the $\bar{a}$ tuples appearing in R, each paired with every $\bar{b}$ tuple from S

   (b) $e_2 \leftarrow (e_1 \setminus R)$ are the $\bar{a}$ tuples from R, each paired with every S tuple which *does not* appear with the respective $\bar{a}$ tuple in R.

   (c) $e_3 \leftarrow \pi_{\bar{a}}(R) \setminus \pi_{\bar{a}}(e_2)$ are all the $\bar{a}$ tuples from R except those which in R *do not appear* with some S tuples. In other words, e_3 are those $\bar{a}$ tuples from R which appear in R with every S tuple, which is exactly R/S.

3. Prove that any relational algebra expression can be equivalently expressed in tuple relational calculus. It can be done by induction on the structure of the relational algebra expression.

4. A query Q is *monotone* iff for any database D_1 and D_2 such that D_1 is included in D_2 then $Q(D_1)$ is included in $Q(D_2)$. Prove that for any query expressed using with the operators $\pi, \sigma, \bowtie, \cup$ and $\times$, this query is monotone. It can be done by induction on the structure of the algebraic expression of the query.

5. Prove that the set of queries expressed by any relational algebra expression is not equivalent to the set of queries expressed using with the operators $\pi, \sigma, \bowtie, \cup$ and $\times$. It suffices to exhibit a non monotone query: $R \setminus S$ for example.

References

[1] Raghu Ramakrishnan and Johannes Gehrke. *Database management systems (3rd ed.)*. McGraw-Hill, 2003.